Coding a Perceptron From Scratch

A page-by-page lecture connecting the math to the Python

This lecture walks through building a working perceptron in Python, one idea at a time. Every page pairs a short explanation with the exact code it produces, so you can see precisely which line of math becomes which line of code -- and why.

Prepared for Gavy | Foundations of Neural Networks series

What Is a Perceptron?

A perceptron is the simplest possible artificial neuron. It takes in one or more numeric inputs, weighs each one by how important it is, adds them together, and makes a single yes-or-no decision based on the total.

That's the whole idea. No hidden magic -- just a weighted vote followed by a threshold check. Every input gets multiplied by a number that says how much it matters (its weight), those products get summed, and if the sum clears a certain bar, the perceptron 'fires' and outputs 1. Otherwise it outputs 0.

Three moving parts

Inputs (x) -- the raw numbers coming in.
Weights (w) -- one per input, learned during training, controlling how much each input matters.
Bias (b) -- a single extra number that shifts how easy or hard it is to fire overall.

Everything on the pages that follow is just turning this idea into working code: represent x and w as lists of numbers, compute the weighted sum, check it against zero, and write a small rule that adjusts w and b when the perceptron gets something wrong.

The Math You Need (and no more)

Before writing any code, it helps to see the math in its final, compact form -- because that compact form is exactly what the code will compute.

$$z = w \cdot x + b$$

$$\text{output} = 1 \text{ if } z > 0 \text{ else } 0$$

w·x is the dot product: multiply each weight by its matching input, then add up all those products. If you have two inputs, this is just w1·x1 + w2·x2 -- the dot product notation is only a shorthand so the formula doesn't grow longer every time you add another input.

b (the bias) is added on afterward. It doesn't depend on any input -- think of it as a constant nudge that makes the perceptron either more eager or more reluctant to fire, regardless of what x looks like.

A hand-worked example

Say w = (-2, -2) and b = 3, with input x = (1, 0):

$$z = (-2)(1) + (-2)(0) + 3 = 1$$

$$1 > 0 \rightarrow \text{fires (output = 1)}$$

That's the entire computation a perceptron ever does. Everything on the next page is simply this arithmetic, written in Python instead of by hand.

From Math to Code: Setting Up

In code, w and x are represented as arrays of numbers -- NumPy is used because it can multiply and sum whole arrays in one step, matching the dot product notation almost word for word.

A Perceptron is built as a small class: a container that bundles its own data (weights, bias) together with the actions it can perform (predict, train). This is a standard pattern in real software -- group the state and the behaviour that acts on that state in one place.

```
import numpy as np

class Perceptron:
    def __init__(self, num_inputs, learning_rate=0.1):
        # Start with small random weights and a zero bias.
        # These numbers will be adjusted as the perceptron learns.
        self.weights = np.random.randn(num_inputs) * 0.1
        self.bias = 0.0
        self.learning_rate = learning_rate
```

__init__ runs once, when a Perceptron object is first created. It doesn't compute anything yet -- it just sets up the starting values that later methods (predict, train) will read and update.

Coding the Forward Pass

The 'forward pass' is just the arithmetic from page 3, written as a method. Given an input x, it returns the perceptron's decision.

```
    def predict(self, x):
        """Forward pass: z = w.x + b, then the step function."""
        z = np.dot(self.weights, x) + self.bias
        return 1 if z > 0 else 0
```

Line by line

np.dot(self.weights, x) computes the dot product w·x in a single call -- this is the entire weighted sum from the equation, done for you.

+ self.bias adds b, exactly as in z = w·x + b.

return 1 if z > 0 else 0 is the step function -- the hard yes/no decision. This one line is the entire 'neuron' part of a perceptron.

At this point the perceptron can already make decisions -- it just doesn't yet know good weights to use. That's what training solves.

The Learning Rule (the math)

Training means adjusting w and b so the perceptron's predictions match the correct answers. The rule only reacts to mistakes:

```
w  =  w + (learning_rate) × (y - y_hat) × x

b  =  b + (learning_rate) × (y - y_hat)
```

y is the correct answer (the label). y_hat is what the perceptron actually predicted. (y - y_hat) is the error:

• If the prediction was correct, y - y_hat = 0, and nothing changes at all.
• If the perceptron should have fired but didn't (y=1, y_hat=0), the error is +1, and the weights get nudged in the direction of x -- making it more likely to fire on similar inputs next time.
• If it fired but shouldn't have (y=0, y_hat=1), the error is -1, and the weights get nudged away from x.

The learning_rate is a small number (like 0.1) that controls how big each correction is -- large enough to actually learn, small enough not to overreact to a single mistake.

Coding the Training Loop

The training loop is the rule from the last page, applied repeatedly over every example, for several passes (epochs) through the data.

```
    def train(self, training_inputs, labels, epochs=20):
        for epoch in range(epochs):
            total_errors = 0
            for x, y in zip(training_inputs, labels):
                x = np.array(x)
                y_hat = self.predict(x)
                error = y - y_hat

                if error != 0:
                    self.weights += self.learning_rate * error * x
                    self.bias += self.learning_rate * error
                    total_errors += 1

            if total_errors == 0:
                print(f"Converged after {epoch + 1} epochs.")
                break
```

What each loop is doing

The outer loop (epochs) repeats the whole training set multiple times -- one pass usually isn't enough to fully correct the weights.

The inner loop (for x, y in ...) goes through every training example one at a time, calling predict() and comparing it to the true label y.

if error != 0 is the entire learning rule from page 6 -- only applied when the perceptron got that example wrong.

if total_errors == 0: break stops training early once a full pass produces zero mistakes -- the perceptron has converged and further training would be wasted effort.

Putting It Together: Learning NAND

With predict() and train() written, the perceptron can now learn a function from raw examples -- instead of being hand-given weights like on page 3, it discovers them itself.

```
inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
labels = [1, 1, 1, 0]          # this is the NAND truth table

p = Perceptron(num_inputs=2)
p.train(inputs, labels, epochs=50)

for x, y in zip(inputs, labels):
    prediction = p.predict(np.array(x))
    print(x, "-> predicted:", prediction, " expected:", y)
```

Notice the training data itself is nothing more than the NAND truth table you already worked out by hand several pages ago. The perceptron isn't told the rule for NAND -- it only sees examples of inputs and correct outputs, and the training loop searches for weights that fit all of them.

Running It: What the Output Means

Running the code above produces this, from actual random starting weights:

```
Converged after 6 epochs.
Learned weights: [-0.18 -0.07]   bias: 0.20

x1  x2 | expected | predicted
 0   0 |        1 |         1   (correct)
 0   1 |        1 |         1   (correct)
 1   0 |        1 |         1   (correct)
 1   1 |        0 |         0   (correct)
```

A few things worth noticing here. First, the learned weights (-0.18, -0.07, bias 0.20) are not the same numbers used on page 3 (-2, -2, bias 3) -- yet both are correct. There isn't one right answer for w and b, only a whole region of values that happen to separate the 1s from the 0s correctly.

Second, the bias came out positive (0.20), matching what you'd expect for NAND -- it fires for three out of four input combinations, so the perceptron needed a bias that makes firing the 'default' behaviour, only switched off when both inputs are strongly on.

Third, 'converged' means a full pass through all four examples produced zero errors -- the training loop's stopping condition from page 7.

The Limit: Why XOR Fails

Swap the labels for XOR's truth table and run the exact same code -- nothing about predict() or train() changes:

```
labels = [0, 1, 1, 0]   # XOR truth table
p = Perceptron(num_inputs=2)
p.train(inputs, labels, epochs=50)
```

No 'Converged' message ever prints, no matter how many epochs you allow. The output keeps getting at least one example wrong forever:

```
x1  x2 | expected | predicted
 0   0 |        0 |        1   (WRONG)
 0   1 |        1 |        1   (correct)
 1   0 |        1 |        0   (WRONG)
 1   1 |        0 |        0   (correct)
```

This isn't a bug and it isn't fixable by tuning the learning rate or training longer. A perceptron's decision boundary is always one straight line (or flat plane in higher dimensions). XOR's two 'fire' points and two 'don't fire' points sit diagonally opposite each other -- no single straight line can ever separate them.

This is the geometric fact -- proved with pen and paper earlier in this series -- showing up as a concrete, reproducible failure in running code. It's the reason multi-layer networks exist at all.

How the Code Is Structured (and why)

Stepping back from the perceptron itself, this file is a small example of a pattern you'll see in almost every piece of real software: separating a system into distinct, single-purpose pieces.

State -- the numbers that define this perceptron's current behaviour (weights, bias). Stored on the object itself (self.weights, self.bias) so every method can read and update the same values.

A pure computation -- predict() takes an input and returns an output, based only on the current state. It doesn't change anything; given the same weights and the same x, it always returns the same answer.

A mutation -- train() is the one method allowed to change the state, and it does so through a single, well-defined rule, applied over and over.

A driver -- the code on page 8 that creates a Perceptron, feeds it data, and reports results. This part knows nothing about how predict() or train() work internally -- it just calls them.

This separation is why you can swap NAND's labels for XOR's on page 10 and change nothing else -- the class doesn't care what function it's learning, only how to learn from whatever examples it's given. Well-structured code stays reusable for exactly this reason.

What's Next

You now have a perceptron that learns from data, and you've seen -- both on paper and in running code -- exactly where its power runs out.

The next two ideas close that gap

1. Sigmoid neurons. Replace the hard step function with a smooth S-shaped curve. This alone doesn't solve XOR, but it makes the neuron's output 'nudgeable' -- a small weight change produces a small output change, which is what makes gradient-based learning possible at all.

2. Hidden layers. Chain multiple perceptron-like units together, the same way you wired NAND gates into a half-adder earlier. Two layers of simple linear units, stacked, can solve problems -- like XOR -- that no single layer ever can.

Your concrete next exercise: extend perceptron.py into a 2-layer network -- two perceptrons in a hidden layer, feeding into one output perceptron -- and get it to solve XOR by hand-picking weights first (the way you did for NAND), before trying to train it automatically. Seeing it work by construction, before it works by learning, is the fastest way to build real intuition for why depth matters.

End of lecture. The full working code from this lecture is available as perceptron.py.